Lazy-LLC: Reducing GPU LLC Thrashing with Shared-Data and Reusability Filters

Sina Darabi[1] Ahmad Javadi-Nezhad[2] Hajar Falahati[1] Negin Mahani[1] Adrian Cristal[1] Osman Sabri Unsal[1]
Hamid Sarbazi-Azad[3]
[1]*Barcelona Supercomputing Center (BSC)*, Barcelona, Spain
[2]*Stony Brook University*, New York, USA
[3]*Sharif University of Technology and Institute for Research in Fundamental Sciences (IPM)*, Tehran, Iran

***Abstract*—Graphics Processing Units (GPUs) are engineered for massive parallelism, supporting high-computation applications, such as deep learning and graphics rendering. The Last Level Cache (LLC) in modern GPUs is pivotal in managing off-chip memory requests. However, large-footprint GPGPU applications often exhibit substantial LLC pressure, leading to wasteful cache occupancy and unnecessary off-chip memory traffic. Despite increases in LLC sizes in recent GPU generations, managing LLC pressure remains challenging because application input datasets have grown proportionally, leading to a larger memory footprint relative to the available LLC capacity. Existing GPU LLC management techniques mainly rely on throttling, static bypassing, or separate prediction structures, often addressing cache pressure without explicitly distinguishing which cache blocks are likely to provide future benefit. In contrast, we propose Lazy-LLC, a unified framework for value-aware LLC admission for large-footprint GPGPU applications that selectively admits data based on its expected reuse and potential for inter-SM sharing. Data with sufficient potential for reuse or sharing are referred to as LLC-friendly data throughout this paper. Lazy-LLC combines complementary runtime signals derived from existing GPU structures, namely the LLC MSHRs and L1D cache behavior, to identify LLC-friendly data while requiring only minimal hardware overhead. Lazy-LLC increases effective bandwidth by 32.54%, reduces the LLC miss rate by 27.03%, and achieves 27.38% higher performance and 21.67% better energy efficiency compared to the NVIDIA A100 GPU in memory-bound applications, with less than 1% increase in thermal design power (TDP) and 0.01% additional die area.**

***Index Terms*—GPU architecture, last-level cache, cache thrashing, cache bypassing, MSHR, memory hierarchy, energy efficiency**

I. Introduction

Graphics Processing Units (GPUs) are designed to handle massive parallelism, making them well-suited to accelerate applications with high computation demands, such as deep learning, scientific computing, cryptography, and graphics rendering [1]. This computation capability is powered by the support of memory hierarchy, introducing a two-level cache, in which each Streaming Multiprocessor (SM) has its own local L1 data cache (L1D) and all SMs share a global last-level cache (LLC) [2].

The LLC in GPUs plays a crucial role in managing off-chip memory requests and total data movement, aiming to reduce the performance bottlenecks that can arise from accessing high-latency off-chip memory [3]. However, excessive memory requests in large-footprint GPGPU applications increase the pressure on the LLC, leading to frequent data eviction—known as thrashing [4]. This condition leads to two significant issues: a reduction in cache hits [5] and an increase in the off-chip memory writes [6].

To reduce LLC pressure, the industry increases LLC sizes across different GPU generations. This trend is evident from Volta [7] (6 MB) to Ampere [8] (40 MB) [3]. However, as LLC sizes increase, application datasets also grow proportionally, leading to the same issue but on a larger scale [9]. Meanwhile, several prior studies focus on balancing LLC pressure. For instance, Zhao et al. [4] propose switching between shared and private LLC organizations dynamically. In another study [10], they suggest fine-grained replication of read-only shared data across LLC slices. Nevertheless, these studies primarily concentrate on balancing LLC pressure without reducing it. Other studies have explored approaches like throttling and bypassing to reduce cache pressure. For example, Darabi et al. [11] enable GPU cores to operate in two modes: a compute mode for executing threads and a cache mode to extend the LLC. Additionally, several studies focus on bypassing the L1D cache. Li et al. [12], for instance, propose a compile-time framework for dynamically adjusting L1D cache bypassing to balance cache usage with application demands. While their work also evaluates bypassing strategies for the LLC (L2 cache), this aspect is explored in a more limited fashion compared to their primary focus on L1D. Therefore, despite these efforts, the potential benefits of bypassing techniques specifically for alleviating LLC pressure remain largely unexplored.

To explore the potential benefits of selectively bypassing the LLC, we measure the LLC hit rate after loading high-locality data (shared and reusable data), termed "LLC-friendly data" in this paper, into the LLC while bypassing the LLC for other data. Our observations indicate that the LLC hit rate is improved by up to 39.78%. To identify LLC-friendly data, we utilize the LLC MSHR and the L1D cache. Our analysis shows that the LLC MSHR tracks 28.62% of shared data among SMs. Additionally, the L1D cache identifies 67.58% of reusable evicted data within SMs for short distances from 1 to 4096 cycles, which is currently discarded by the GPU's cache policy [2].

To address the challenges posed by large-footprint GPGPU applications, we propose Lazy-LLC, a lightweight LLC admission framework that identifies LLC-friendly data using complementary runtime signals derived from existing GPU structures. Rather than admitting every off-chip memory request into the LLC, Lazy-LLC selectively admits only data with sufficient reuse or inter-SM sharing potential, reducing unnecessary LLC occupancy while requiring only minimal hardware overhead.

Lazy-LLC classifies off-chip memory requests into two categories. The first includes requests whose locality is explicitly known by the GPU, such as atomic operations and texture accesses, which continue to follow the baseline LLC policy. The second includes requests whose locality is initially unknown. For these requests, Lazy-LLC performs admission in two stages. First, it determines whether the requested data are shared across multiple SMs using the LLC MSHRs. If no sharing is detected, the request initially bypasses the LLC and is cached only in the L1D cache. When the block is later evicted from L1D, its reuse behavior is evaluated. Only blocks identified as reusable are subsequently admitted into the LLC, where they remain available for future accesses.

To evaluate the effectiveness of Lazy-LLC, we conduct experiments using the AccelSim [13] simulator on NVIDIA A100 GPU [8]. Our experimental results demonstrate that Lazy-LLC significantly increases effective bandwidth by 32.54% and reduces the LLC miss rate by 27.03%, which translates to substantial gains in overall performance by 27.38% and energy efficiency by 21.67%, compared to the baseline in Memory-Bound applications. Additionally, Lazy-LLC outperforms state-of-the-art throttling techniques [11] and bypassing strategies [12], achieving IPC improvements of 11.64% and 14.41%, respectively. Notably, these benefits come with minimal overhead, incurring less than 1% increase in thermal design power (TDP) and 0.01% additional area.

This paper makes the following contributions:

- We show that modern GPUs experience high LLC pressure leading to cache thrashing, and identify two types of LLC-friendly data: shared data across SMs and reusable data within an SM with short reuse distances.
- We propose Lazy-LLC, a novel cache management approach, that leverages the LLC MSHR's ability to detect data sharing among SMs and the L1D's ability to detect reusable evicted data within SMs to reduce LLC pressure effectively.
- We demonstrate that Lazy-LLC reduces writeback traffic to off-chip memory by employing a lazy insertion policy in the LLC, lowering the insertion rate and thereby decreasing the number of writes to main memory.
- We demonstrate the substantial benefits of Lazy-LLC, including a 32.54% increase in effective bandwidth, a 27.03% reduction in LLC miss rate, a 27.38% improvement in overall performance, and a 21.67% enhancement in energy efficiency in Memory-Bound applications.
- We compare our approach with state-of-the-art throttling [11] and bypassing [12] techniques, demonstrating IPC improvements of 11.64% and 14.41%, respectively.
- We highlight the minimal overhead of Lazy-LLC, with less than 0.01% die area overhead and 1% thermal design power (TDP) overhead compared to the Ampere A100 GPU [8].

II. Background and Motivation

GPUs evolved from specialized devices for rendering images to powerful, general-purpose processors capable of handling various computational tasks [2]. Their architecture is specifically designed to manage a high level of parallelism, effectively running thousands of threads simultaneously. The efficiency of GPUs in handling parallel operations is largely supported by a memory hierarchy, which is crucial for maintaining high throughput and performance [1].

As illustrated in Figure 1, this hierarchy includes two levels of cache designed to reduce data access latency and enhance computation speed by keeping frequently accessed data closer to the SMs. Each SM in a GPU is equipped with its own L1D cache. The L1D cache is a small, fast cache that stores local data to the threads running on the SM. Above the L1D cache, the LLC functions as a shared storage among all SMs within the GPU, which acts as a buffer between the high-speed on-chip caches (e.g., L1D cache) and the slower off-chip memory [14]. Note that the GPU caches are non-inclusive and non-exclusive, which leads to decreased coherency cost, data replication, and network-on-chip (NoC) traffic.

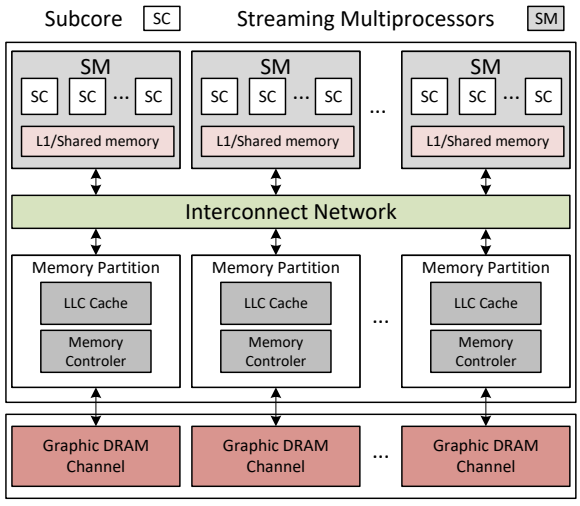


Fig. 1. Baseline GPU architecture.

Due to the intense pressure from thousands of concurrent threads and increasing dataset sizes with irregular access patterns, the LLC frequently experiences data eviction, commonly referred to as thrashing [4]. To alleviate thrashing for modern applications like AI, Nvidia's A100 [8] increases LLC capacity, utilizes LLC compression, and incorporates persistent memory. This highlights the growing dataset sizes in real-world applications and shows that previous programming techniques are insufficient for addressing the thrashing problem.

Contention leads to two significant issues: First, a notable reduction in the LLC hit rate [5], as data is evicted before it

can be reused. Second, an increase in off-chip memory writes due to the frequent eviction of dirty data [6]. When the LLC hit rate decreases, the GPU is forced to access slower off-chip memory more frequently, significantly increasing latency [15]. Additionally, the higher number of off-chip memory writes further exacerbates latency and can lead to increased power consumption and reduced overall system performance [16].

To analyze wasteful LLC occupancy and its impact on hit rate and off-chip memory traffic, we collect offline LLC access traces from various GPGPU applications using the Accel-Sim simulator [13], which models an NVIDIA A100-like GPU [8]. Each trace is collected after a warmup phase of 500,000 instructions, using approximately 2 million instructions per application. We use a sampling window of 4096 requests to characterize reuse behavior (see Section IV). High LLC contention in GPUs is driven by thousands of concurrent threads issuing memory requests. To characterize ineffective cache occupancy, we define wasteful caching as the fraction of LLC-inserted blocks that are evicted before providing any reuse benefit. This includes both single-access blocks and blocks whose reuse occurs only after they leave the LLC. As shown in Figure 2, the average wasteful caching rate is 47.89%. This high rate indicates that a large fraction of LLC capacity is occupied by low-utility blocks, increasing cache contention and contributing to higher off-chip memory traffic.

Figure 3 shows that the average LLC hit rate is only 35.60%, with memory-heavy workloads like SAD, SPMV, and LIB performing even worse (10.32%, 21.74%, and 28.09%, respectively). Comparing Figures 2 and 3 confirms that higher thrashing leads to lower hit rates. Frequent evictions of reusable data cause repeated off-chip accesses, slowing performance.

Thrashing also increases the eviction of modified (dirty) data, leading to more off-chip write requests. Figure 4 highlights this impact by showing write requests per kilo instructions (WPKI), with high-thrashing workloads like SAD and SPMV reaching very high WPKI values (176.17, 119.25). The combination of low hit rates and frequent dirty evictions puts extra strain on memory bandwidth and energy efficiency.

These findings emphasize the need to reduce thrashing by keeping frequently reused data in the LLC, as our Lazy-LLC design aims to do.

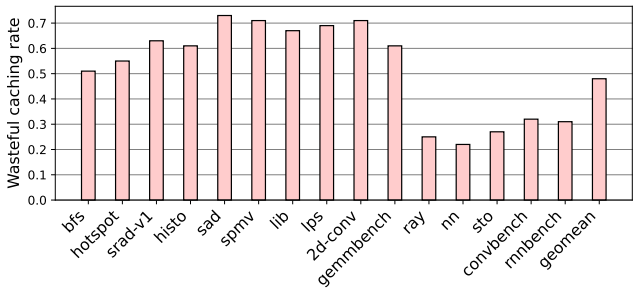


Fig. 2. LLC wasteful caching rate.

To address the pressure on LLCs, the industry has attempted to increase their size across different GPU generations [8], [17]. However, we observe that simply increasing LLC capacity does not effectively resolve the thrashing challenge

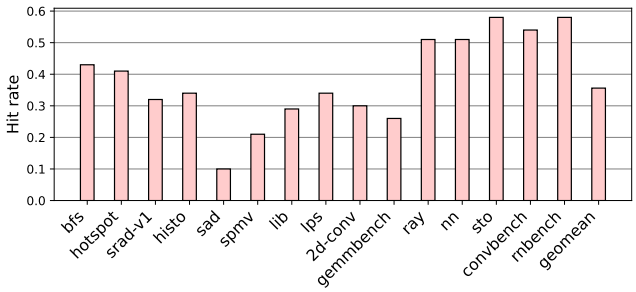


Fig. 3. LLC hit rate.

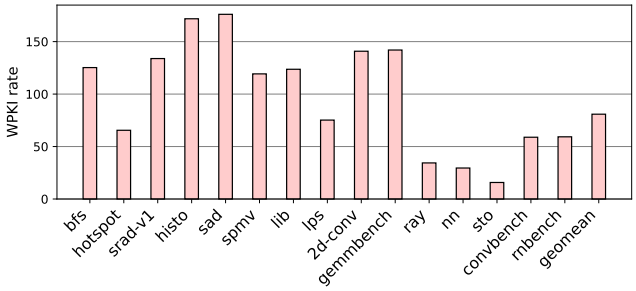


Fig. 4. Number of off-chip write requests per kilo instructions.

(47.89% of LLC data are evicted before being reused see Figure 2). This is because modern applications have large primary working sets that often exceed modern LLC capacity. Additionally, these applications typically exhibit irregular access patterns [3]. Prior studies have explored alternative techniques to mitigate LLC pressure, such as balancing [4], [10] or directly reducing the pressure through throttling [11]. However, prior work mostly focuses on bypassing the L1D cache, and there is limited work investigating the potential of optimizing LLC pressure through a bypassing strategy [12].

Therefore, we aim to reduce pressure directly by loading LLC-friendly (shared and reused) data into the LLC. Our observation indicates that existing GPU components, including LLC MSHR and L1D cache, can identify shared data among SMs and reusable data within SMs (i.e., LLC-friendly data), respectively.

In GPUs, the large volume of memory requests often leads to frequent cache misses, making effective tracking and management of these requests critical. LLC MSHRs, which manage off-chip memory requests that cannot be immediately serviced, are key to handling this load efficiently. Therefore, GPUs typically have numerous MSHRs to support this demand. For example, NVIDIA's Volta, Turing, and Ampere architectures each incorporate 192 MSHRs per LLC partition [18]. LLC MSHRs maintain data on memory request status, such as memory addresses and identifiers of requesting units, enabling GPUs to effectively manage and fulfill multiple off-chip requests to the same address [19]. Note that, when multiple off-chip memory requests from various SMs target the same address while an MSHR entry is active, the LLC MSHR can naturally detect that the data is shared among SMs. This existing capability allows us to identify LLC-friendly shared data with no additional hardware cost.

To verify this opportunity, we analyze the entries in MSHRs across various GPGPU applications (see Section IV-B). In this analysis, data is considered shared when an MSHR entry for a specific address is initialized by one SM and new accesses to the same address later arrive from other SMs. As shown in Figure 5, the outer bar shows the percentage of *all* memory accesses that are detected as shared by the MSHR hardware (i.e., accessed by at least two SMs). The inner bar shows the percentage of *all* memory accesses that are shared by more than two SMs **and** are accessed again by other SMs after the initial detection by the MSHR. As observed, 28.62% of all memory accesses are detected as shared through the LLC MSHR, while 15.37% of all memory accesses are shared by more than two SMs and experience subsequent accesses from other SMs after MSHR detection.

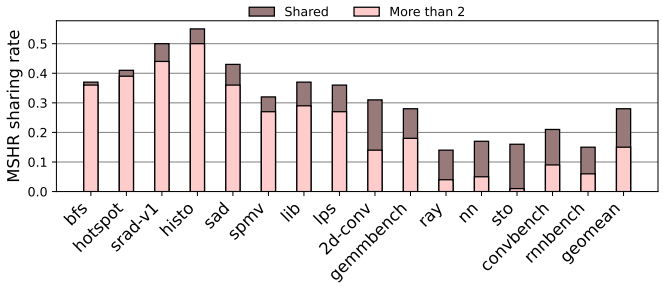


Fig. 5. Shared accesses stored in the MSHR entries.

To study the L1D cache's ability to detect reusable data, we analyze the reuse behavior of L1D-evicted data (within each individual SM). Specifically, we only consider L1D victims that experienced at least one hit before eviction. We define *reuse distance* as the number of requests between two consecutive accesses to the same victim address in the L1D cache of one SM. We then categorize these reuse distances into five intervals: 1–512, 513–1024, 1025–2048, 2049–4096, and 4097–∞ requests. Figure 6 shows the reuse distance distribution of hit-experienced L1D victims per SM. On average, 67.58% of these victims are reused within 4096 requests. Notably, 4096 requests roughly corresponds to the number of cache blocks that can fit in one 512 KB LLC slice (i.e., $512\,\text{KB}/128\,\text{B} = 4096$ blocks). For some applications such as lib, this fraction reaches up to 93.71%. These results indicate that the L1D cache is an effective locality detector: a significant portion of useful data is evicted from the L1D while it still exhibits short-term reuse that fits within the capacity of a single LLC slice. By promoting such data to the LLC upon L1D eviction, Lazy-LLC can reduce thrashing and improve cache efficiency.

In conclusion, our analysis highlights the substantial challenge posed by LLC pressure in GPUs, which leads to high thrashing rates and decreased system performance. Simply increasing LLC size, as seen in industry trends, does not effectively mitigate this issue. However, we identify a promising opportunity to optimize LLC performance by leveraging existing GPU components—specifically the LLC MSHR and L1D cache. These components are capable of detecting shared

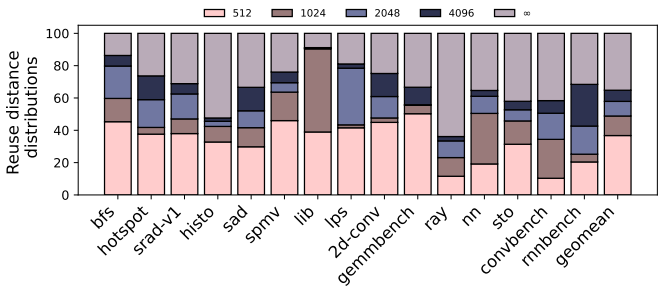


Fig. 6. Reuse distance distribution of the data in L1D cache.

data across SMs and reusable data within individual SMs, respectively. By strategically loading LLC-friendly data into the LLC, we propose a more efficient caching strategy that can reduce thrashing, improve data reuse, and enhance overall GPU performance. This approach opens up new ways for mitigating LLC pressure without requiring significant hardware changes.

III. Lazy-LLC

We introduce Lazy-LLC, an innovative cache management technique that leverages existing GPU components to identify the most suitable data for placement in the LLC, aiming to reduce LLC pressure directly and enhance overall system performance effectively.

Figure 7 shows the data caching flow in Lazy-LLC. In Lazy-LLC the data is cached in LLC in four ways: ① the L1D dirty data, ② special instruction such as atomic, fence, and texture, ③ the shared data detected by LLC MSHR (called MSHR filter), and ④ the reused L1D victims (called L1D filter). In this flow, when an SM requires data, the GPU first checks the L1D cache and then the LLC to determine if the data is present in either of these caches. If the data is missing in these caches and needs to be fetched from off-chip memory, the LLC MSHR filter examines the requested data to identify if it is requested by other SMs. If the data is requested by other SMs, it is marked as shared and placed into the LLC. Additionally, if the data is not detected as shared in LLC MSHR, Lazy-LLC bypasses LLC and the data is cached in L1D for processing using the L1D filter. The L1D filter detects the data that is going to be used shortly (i.e., reusable data) and sends it to the LLC, upon eviction.

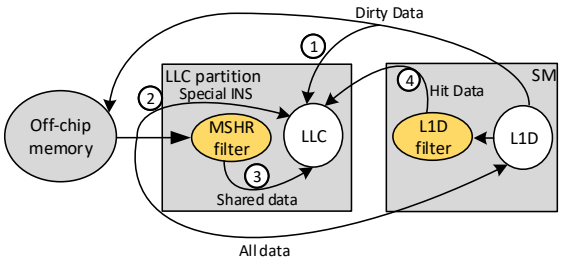


Fig. 7. The data caching flow for Lazy-LLC.

Figure 8 illustrates the proposed modifications to the baseline GPU architecture. Lazy-LLC introduces lightweight

changes to two components of the baseline GPU: the LLC MSHR and L1D cache. Lazy-LLC suggests using the conventional MSHR in each LLC partition with modified Sharing Checker hardware as a filter ① to detect shared data among SMs and employs a modified L1D as another filter with a reusability identifier hardware ② to examine potential reusable data candidates for LLC caching among L1D cache victims. Notably, both filters can operate independently to enhance LLC performance by improving the detection of LLC-friendly data. In the following sections, we present a detailed introduction to the LLC MSHR filter (Section III-A) and the L1D filter (Section III-B).

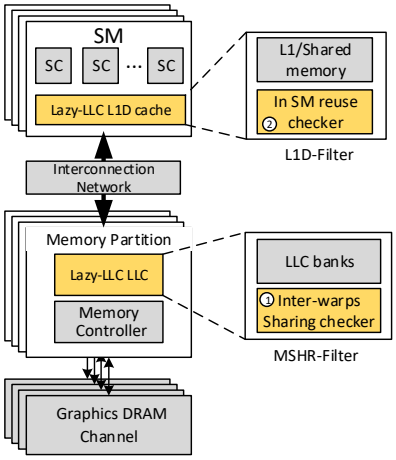


Fig. 8. GPU architecture including L1D filter and MSHR filter.

A. MSHR-based Sharing Filter

Figure 9 illustrates the baseline LLC MSHR organization. Each memory partition contains a 192-entry MSHR table, where each entry records the requested block address, request status, and up to four coalesced load/store requests targeting the same cache block. In the baseline design, the MSHR supports non-blocking memory accesses by tracking outstanding LLC misses and coalesces requests from different threads to reduce redundant off-chip memory transactions [2], [8]. These properties make the MSHR a natural point for observing runtime sharing behavior.

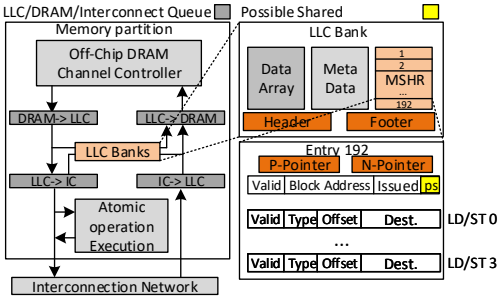


Fig. 9. The MSHR microarchitecture in Lazy-LLC.

Lazy-LLC augments the MSHR-based sharing filter with a lightweight Logical Sharing Tracker. The key idea is to retain the metadata of recently serviced MSHR entries as logical sharing records, instead of immediately invalidating them after the memory response arrives. These records are stored in MSHR entries that would otherwise be free and are managed by a FIFO-based controller. This allows Lazy-LLC to detect later requests to the same block and identify inter-SM sharing that would be missed by the baseline MSHR.

1) Logical Sharing Tracker Design and Operation: After an MSHR entry completes service, i.e., when the memory response returns, the MSHR would normally free the entry. Lazy-LLC instead temporarily retains the entry metadata as a logical sharing record when the returned block is not admitted into the LLC. This record keeps only the information needed to recognize a later request to the same cache block; it does not store the cache block data. These logical sharing records are embedded inside otherwise free MSHR entries and managed by a FIFO-based controller. When a serviced entry is converted into a logical sharing record, Lazy-LLC sets its Possible Shared (PS) bit and links the entry into the FIFO-managed tracker. This allows Lazy-LLC to detect later requests to the same block, even when no active MSHR entry exists for that address.

The Logical Sharing Tracker is implemented using free MSHR entries and therefore requires no separate storage array. It extends each MSHR entry with two 8-bit pointers (`prev` and `next`) and a 1-bit Possible Shared (PS) flag. Two 8-bit registers (Head and Tail) implement a FIFO-based controller that manages the logical sharing records. Each record stores only metadata, including the block address, the PS bit, and the list pointers; cache block data is never stored. A logical sharing record remains until either a later request matches the same block or the corresponding MSHR entry is needed to service a new miss. When no free MSHR entry is available, the FIFO-based controller recycles the oldest logical sharing record and reuses its entry for the incoming miss request. This policy preserves recent sharing history while ensuring that the tracker occupies no additional storage beyond the existing MSHR table.

Upon each LLC miss, the MSHR performs its associative lookup over both active MSHR entries and logical sharing records. If the requested block matches a logical sharing record, the corresponding entry is removed from the tracker and reactivated as a normal MSHR entry. The incoming request is merged into this entry, the PS bit remains asserted, and inter-SM sharing is confirmed. Consequently, the returned cache block is admitted into the LLC. The Logical Sharing Tracker does not store or forward cache block data. On a tracker match, the data still arrives through the normal memory response path. The tracker only preserves recent address-level metadata to identify inter-SM sharing and guide the LLC admission decision. Since a matched record corresponds to a previously completed request, the later request provides stronger evidence of repeated access to the same block after the previous miss has completed than relying only on active in-flight MSHR coalescing.

The tracker lookup adds at most two cycles. This overhead

occurs only on LLC misses and is not on the critical path, since the memory request can be issued immediately while the tracker lookup and bookkeeping are performed in parallel.

2) Illustrative Example: Consider memory block A:

1) At time T1, SM1 misses on address A and allocates an active MSHR entry.
2) At time T2, the memory response returns. Since the block is not admitted into the LLC, Lazy-LLC converts the serviced MSHR entry into a logical sharing record by setting the PS bit and linking the entry into the FIFO-managed tracker.
3) At time T3, SM2 requests address A. Although no active MSHR entry exists for A, the Logical Sharing Tracker matches the block address and confirms a later request from another SM.
4) At time T4, the matched record is reactivated as an MSHR entry, and the new request is merged into it. When the data returns through the normal memory response path, the block is admitted into the LLC.

Figure 10 shows the contribution of the Logical Sharing Tracker to inter-SM sharing detection across the evaluated benchmarks. The PS FIFO contributes between 13% and 51% of the detected inter-SM sharing, with a geometric mean contribution of 28%. This result highlights that a substantial portion of sharing opportunities are captured only after the baseline MSHR entry is freed, demonstrating the effectiveness of retaining recently serviced MSHR metadata in improving the overall sharing detection capability of Lazy-LLC.

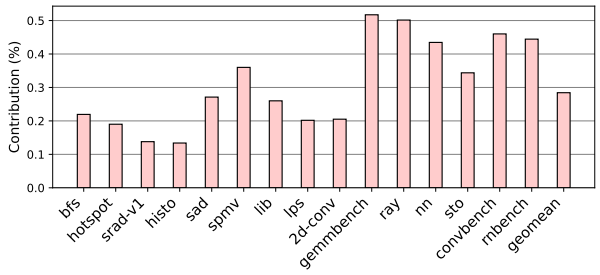


Fig. 10. PS FIFO Sharing Detection Contribution (%)

The complete MSHR miss-handling flow in Lazy-LLC is shown in Figure 11. On an LLC miss, the MSHR performs an associative lookup over both active MSHR entries and logical sharing records. If the request matches a record with the PS bit set ①, the entry is reactivated as an MSHR entry, the new request is merged into it, and the returned block is admitted into the LLC ②. If the request matches an active MSHR entry with an available load/store slot ③–④, it is coalesced normally. If all load/store slots are occupied, the request is retried until a slot becomes available ⑤. When no matching entry is found ⑥ and the MSHR has free capacity ⑦, a new active entry is allocated. If the MSHR is full ⑧ but the Logical Sharing Tracker is not empty ⑨, the FIFO-based controller recycles the oldest logical sharing record and reuses that entry for the incoming miss ⑩. This recycling policy preserves recent sharing history while avoiding additional storage beyond the existing MSHR table.

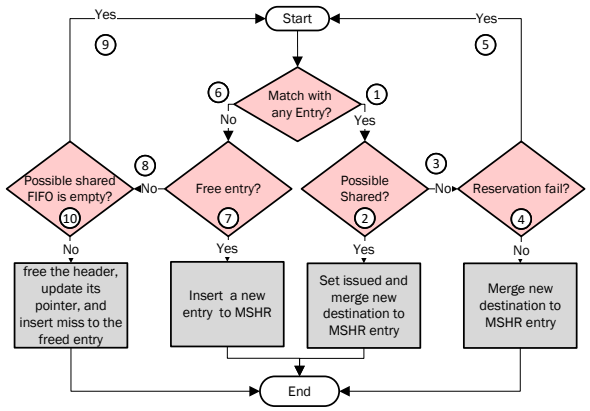


Fig. 11. The flowchart of the proposed MSHR miss handling in Lazy-LLC.

B. L1D Filter

In Lazy-LLC's LLC cache flow, as illustrated in Figure 7, once data bypasses the LLC, it indicates that the LLC MSHR did not detect it as shared. Consequently, in addition to serving the requesting thread at the SM level, the data is subjected to assessment for its possible reusability upon eviction from the L1D cache. To detect reusable data, Lazy-LLC introduces a new filter powered by the L1D cache in each SM, utilizing dynamic runtime access information for each block to determine the reusability of evicted data.

Figure 12 illustrates the L1D filter, comprising three main units: the victim unit, the threshold unit, and the decider unit. The victim unit acts as a buffer, temporarily storing evicted data until the decider unit assesses its eligibility for LLC caching. The threshold unit, a 2-bit register, tracks the number of data accesses required for caching in the LLC. The decider unit makes the final LLC caching decisions: it compares the threshold with the hit-counter bits (C0 and C1), decides whether to cache the victim data in the LLC, removes redundant metadata, and transmits the final data through the interconnection network.

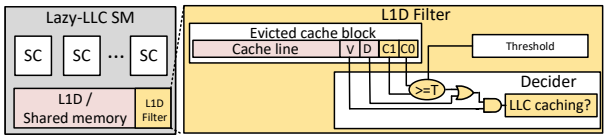


Fig. 12. Overall view of L1D Filter in Lazy-LLC.

Lazy-LLC extends each data block in the L1D cache with two bits (C0 and C1) that record its four most recent accesses (implemented as a 2-bit saturating counter). An additional dirty bit (D) indicates whether the block has been modified, and a valid bit (V) tracks validity. Lazy-LLC does not consider dirty data for LLC insertion, since the GPU baseline already handles dirty writebacks to higher memory levels correctly. The complete decision logic of the L1D filter is formalized in Algorithm 1.

The proposed functionality of the L1D filter is also depicted in the flowchart in Figure 13. When data arrives at the L1D

Algorithm 1 L1D Filter Operation in Lazy-LLC

Require: Evicted block from L1D cache
Ensure: Decision: insert into LLC or evict without caching

Global: THRESHOLD ← T1 (binary 01) {Fixed threshold selected via sensitivity study (Fig. 14); T1 provides the best IPC without runtime overhead}
Extended L1D block metadata per line: V (valid bit), D (dirty bit), C0/C1 (2-bit hit counter for up to 3 recent accesses)
victim ← evicted_block
if victim.V = 0 **then**
    evict without LLC caching
    return
end if
if victim.D = 1 **then**
    handle dirty writeback to higher memory level (baseline GPU policy)
    invalidate victim in L1D
    return {Lazy-LLC does not consider dirty data for LLC insertion}
end if
access_count ← (victim.C0 ≪ 1) ∨ victim.C1 {Value in range 0–3}
threshold_val ← decode(THRESHOLD) {T0 ↦ 0, T1 ↦ 1, T2 ↦ 2, T3 ↦ 3}
if access_count ≥ threshold_val **then**
    remove redundant metadata bits from victim
    transmit victim to LLC via interconnection network
    return {LLC-friendly reusable data is inserted}
else
    evict without LLC caching
    return
end if

filter, it first inspects the valid bit of the evicted block ①. If the block is invalid, the filter ignores it and the block is evicted without LLC caching ②. If the block is valid, the filter checks the dirty bit ③. For clean (non-dirty) blocks, the decider unit evaluates reusability by comparing the hit-counter bits (C0 and C1) against the threshold ④. If the block meets the reusability criterion, its metadata bits are stripped and the data is transmitted to the LLC via the interconnection network ⑤. Otherwise, the block is simply evicted without LLC caching ⑥. **Threshold Selection and Design Rationale.** The effectiveness of the L1D filter depends on the choice of threshold. Since the 2-bit hit counter can record at most three accesses per block, we exhaustively evaluated all four possible threshold values (T0–T3) and measured end-to-end GPU performance (Figure 14).

T1 (binary 01) delivers the highest average IPC improvement. Consequently, Lazy-LLC adopts a **fixed global threshold of T1** for all applications at runtime. We deliberately chose a fixed threshold rather than a dynamic one for two reasons: (1) it eliminates any runtime computation or hardware

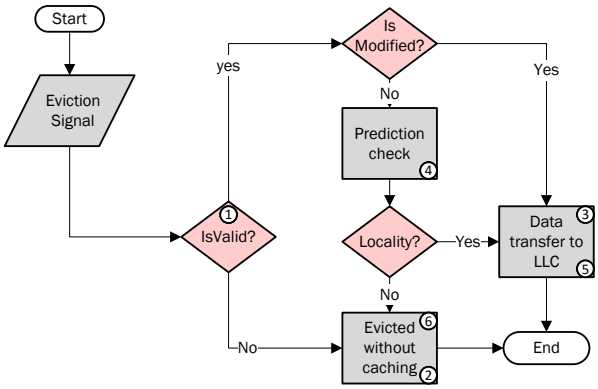


Fig. 13. Flowchart of L1D filter.

overhead, and (2) our sensitivity study shows that T1 is robust across the majority of workloads. A single static value also keeps the L1D filter extremely simple and low-cost, consistent with our goal of minimal hardware modification. T1 was selected by evaluating all four threshold values on the same benchmark suite used in the final evaluation. However, the risk of overfitting is negligible because (i) the design space is tiny (only four discrete values), (ii) we report the complete sensitivity results for every threshold, and (iii) we apply one globally fixed threshold rather than performing per-application tuning. T1 consistently outperforms the other three settings in geometric mean and in most individual applications, indicating a genuinely robust operating point rather than an overfitted choice.

In contrast, higher thresholds (T2 and T3) are overly conservative. They reduce the performance gain from 18.55% down to 7.58% because too few blocks qualify for LLC insertion, leaving the LLC under-utilized. T0, while simple, admits blocks with weak reuse into the LLC and therefore yields the smallest improvement. As discussed earlier, T0 resembles an exclusive-style insertion policy but without cross-SM invalidations; it helps in a few cases by reducing duplication but hurts applications that benefit from retaining temporal locality across the L1D–LLC boundary.

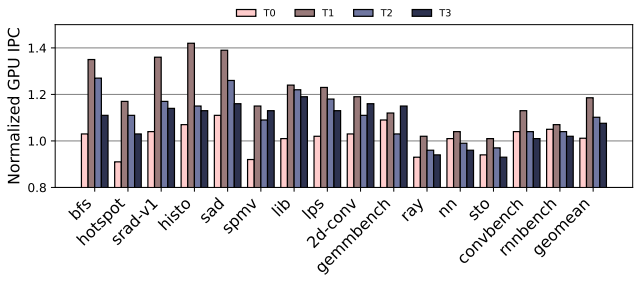


Fig. 14. Total GPU IPC with varying threshold from T0 to T3.

C. Special Operation Execution and Correctness

Lazy-LLC is designed to preserve the original GPU memory consistency model and synchronization semantics. It only modifies LLC admission decisions for regular read misses and does not alter the handling of memory-ordering and

synchronization operations. These operations are identified by their instruction or request type and continue to follow the baseline LLC caching path, thereby preserving their original ordering, visibility, and serialization behavior.

Texture accesses exhibit strong spatial and temporal locality with well-defined access patterns. For example, the `hotspot` and `srad-v1` benchmarks from the Rodinia suite bind temperature maps and image data to texture memory to exploit efficient 2D spatial locality. Lazy-LLC does not modify texture caching behavior. These operations continue to use the baseline texture caching mechanism, ensuring that applications relying on texture memory retain their original performance and correctness.

Fence instructions serve as explicit synchronization points that enforce memory ordering across threads and blocks. Lazy-LLC does not affect fence instructions; they are routed through the baseline LLC path to guarantee correct memory visibility and ordering.

Atomic operations, such as `atomicAdd` and `atomicCAS`, require exclusive access to memory locations to ensure correct inter-thread communication and to prevent data races. Lazy-LLC does not interfere with atomic operations. They continue to follow the baseline LLC caching path to preserve atomicity and ordering semantics. For instance, the `histo` benchmark relies heavily on atomic instructions to safely update shared histogram bins across many threads. By keeping atomic operations on the baseline path, `histo` benefits from the performance improvements of Lazy-LLC on regular memory accesses without any loss of functionality or correctness.

Lazy-LLC does not affect memory consistency enforcement. All consistency-related operations follow the baseline LLC behavior, thereby preserving the original GPU memory consistency model. Furthermore, Lazy-LLC maintains the original non-inclusive, non-exclusive GPU cache hierarchy and does not introduce new invalidations or modify coherence protocols.

Finally, Lazy-LLC does not affect context switch behavior. During GPU context switches, all baseline consistency and coherency mechanisms remain unchanged. Since Lazy-LLC does not modify dirty writeback, flush, or invalidation policies, the existing GPU context switch mechanism continues to function correctly with no additional overhead or handling required.

D. Lazy-LLC Impacts

1) Memory Access Latency:: The key decision made by Lazy-LLC is whether to cache data in the LLC or not. This decision does not impact L1D cache or LLC hit latency, which remains around 20 and 100 cycles, respectively. However, it may increase cache miss latency by up to 2 cycles due to the PS FIFO update operation. This minor increase is negligible compared to the DRAM access latency which is 400 cycles.

While Lazy-LLC may slightly increase the latency of some memory accesses like writes, due to more victim traffic from L1Ds cache to LLC, it only decides about read instructions in the LLC. Writes are treated the same as the baseline. During kernel context switches, there is no read instruction execution or eviction due to reads, so Lazy-LLC does not impact latency.

In summary, our approach increases the maximum memory access latency by at most 2 cycles. However, it reduces the average effective memory access latency by improving the LLC hit rate. The overall performance improvement decreases the impact of memory operations on latency by reducing off-chip memory requests. Notably, read instructions, the target of Lazy-LLC, make up the majority (over 90%) of memory accesses in memory-bound GPGPU applications.

2) Interconnection network traffic tuning:: Transferring reusable victim blocks to LLC amplifies the traffic on the interconnection network, potentially leading to performance degradation due to the increased pressure. However, under such circumstances, our mechanism tunes the system's performance inherently. When the performance decreases, it results in fewer read accesses to the L1D and consequently, fewer victim blocks are dispatched to the LLC. This natural adaptation reduces the burden on the interconnection network (see Section V-C and Section V-D for more details).

E. Cost Analysis

Lazy-LLC adds the following overhead if the threshold always is 1: (1) a decider unit for L1D cache, (2) 1 bit per L1D cache block for storing hit status, (3) a modified MSHR controller, (5) 17 extra bits per MSHR entry (1-bit PS for prefetch status, 8-bit next pointer, and 8-bit previous pointer), and (6) two 8-bit registers for Header and Footer pointers for each MSHR unit. Note that 8 bits are sufficient for encoding the 192 entries of the baseline MSHR in each LLC bank.

Area. We implement Lazy-LLC units in Hardware Description Language (HDL) and synthesize them using Synopsys Design Compiler in 28nm technology, scaled to 12nm (our baseline technology). We then conduct place and route using Cadence SoC Encounter. Moreover, we model the memory components using CACTI V7.0 [20] for 22nm technology and scale the extracted area to the 12nm process node. Our analysis reveals that Lazy-LLC imposes a negligible area overhead (less than 0.01%) compared to the baseline GPU architecture.

Power. We synthesize the HDL model of all added resources using the NanGate 28nm open cell library with Synopsys Design Compiler, scaling the results to the 12nm technology node. We then import the calculated values into AccelWatch [13]. Our analysis shows that Lazy-LLC imposes a negligible thermal design power (TDP) overhead (up to 1%) compared to the baseline GPU architecture.

IV. Methodology

A. Simulation

We evaluate Lazy-LLC using Accel-Sim V1.2.0 [13] and AccelWattch V1.0 [21], modeling a Ampere-like GPU [8]. While the proposed concept is versatile enough to be implemented across various GPU generations, our focus is on the Ampere A100 model as it's the latest generation with advanced architectural features. Table I shows the details of our simulation parameters. We modify different parts

of Accel-Sim (e.g., the front-end, the decode stage, and the execution units) to provide the architectural support of Lazy-LLC (see Section III for more details).

TABLE I
SIMULATION CONFIGURATION.

Parameter	Value
Number of SMs	108
Core clock	1410 MHz
Scheduler	Greedy Then Oldest (GTO)
Number of schedulers	4 per SM
Number of warps	64 per SM
Register file size	256 KB per SM
Shared memory size	64KB per SM
Unified L1D cache	adaptive, 192KB, 64-way, 128B-line 6-bank, Write-Evict policy
L1 instruction cache	128KB, 16-way, 128B-line
LLC	512 KB per memory sub-partition 64-way, 128B-line, 80-Slice, 40 MB 192-MSHR entry (entry-length 4) Write-Back policy
Cache inclusion policy	Non-inclusive non-exclusive (NINE)

B. Benchmarks

We evaluated Lazy-LLC using diverse applications from PolyBench [22], Rodinia [23], Ispass2009 [24], Parboil [25], and DeepBench [26], [27] (Table II). Our benchmark selection focuses on diversity across HPC and AI domains, reproducibility through parameter-configurable inputs, and the ability to stress modern GPU hardware resources.

To evaluate Lazy-LLC under high memory pressure, we use enlarged dataset configurations, following a methodology adopted in recent GPU architecture studies. Works such as ALTIS [28] and Cactus [29] similarly employ stressed or scaled inputs to better exercise new GPU features, including large last-level caches and high-bandwidth memory systems. This approach allows us to analyze Lazy-LLC under demanding conditions with strong data locality and high off-chip bandwidth demand [9], [30]. Table II also indicates which filtering mechanism (MSHR or L1D) is most effective for each application. The results demonstrate that Lazy-LLC performs particularly well under these challenging conditions.

C. Comparison Points

To demonstrate the effectiveness of Lazy-LLC, we compare it against six configurations: (1) the baseline GPU, (2) Lazy-LLC$_{\text{MSHR}}$, which uses only the MSHR filter, (3) Lazy-LLC$_{\text{L1D}}$, which uses only the L1D filter, (4) Morpheus [11], (5) AdaptCache [12], and (6) Belady-B.

Morpheus was re-implemented by modifying the resource allocator and LLC controller to dynamically extend effective LLC capacity using idle register files and shared memory, following the original design. AdaptCache was implemented as a compile-time bypassing framework using the same cost-benefit model and decision thresholds as described in the original paper, with parameters re-tuned for our simulation setup.

Belady-B is implemented as an oracle that has access to future memory access traces within each LLC set. It makes optimal bypass and insertion decisions by always selecting the block that will be referenced farthest in the future, following the principle of Belady's MIN algorithm adapted for cache bypassing. Belady-B represents the theoretical upper bound on performance achievable by any bypassing or insertion policy and is used solely as a comparison point to evaluate how close Lazy-LLC comes to the ideal case.

All experiments were conducted using the Accel-Sim [13] framework modeling an NVIDIA Ampere-like GPU. Memory traces were collected after a warmup phase of 500K instructions, followed by the simulation of approximately 2 million instructions per application. For the reuse distance analysis in Figure 6, we used a sampling window of 4096 requests, and all reported results are arithmetic means across the evaluated applications.

D. Comparison Metrics

To thoroughly evaluate Lazy-LLC, we assess our approach using several key metrics: LLC miss rate, LLC bandwidth efficiency, interconnection network traffic, off-chip memory accesses, performance improvement (IPC), and energy consumption.

V. EVALUATION

A. LLC Miss Rate Analysis

Figure 15 illustrates the LLC miss rate for various implementations, normalized to the baseline. The implementations include Lazy-LLC, Lazy-LLC_MSHR, Lazy-LLC_L1D, Morpheus, AdaptCache, and an ideal case. For memory-bound applications, Lazy-LLC_MSHR and Lazy-LLC_L1D reduce the LLC miss rate by 18.23% and 22.89%, respectively, demonstrating the effectiveness of the L1D cache and LLC MSHR filters in Lazy-LLC for selectively caching reusable and shared data in the LLC, which results in an overall average reduction of 27.03% in the LLC miss rate. Moreover, while AdaptCache offers moderate LLC miss rate reductions of around 15.61%, it is less efficient than Lazy-LLC, particularly under memory-bound workloads, which indicates that Lazy-LLC is able to detect LLC-friendly more efficiently. However, for compute-bound applications, Lazy-LLC does not show improvement, as these applications can load entire datasets into caches, whereas Lazy-LLC only stores portions of datasets identified by the LLC MSHR and L1D filters. It is worth noting that Lazy-LLC outperforms Morpheus, the current state-of-the-art model, reducing the miss rate by approximately 11.4% in memory-bound applications.

Beyond the direct improvement in LLC miss rate, Lazy-LLC also reduces pressure on the LLC MSHR through a virtuous cycle. As shown in Figure 16, Lazy-LLC reduces reservation failures by a average of 6.53% across memory-bound applications. This reduction occurs because the lower LLC miss rate decreases the rate at which new MSHR entries are allocated. With reduced allocation pressure, the PS FIFO can retain recently serviced MSHR entries for a longer effective window without being prematurely overwritten. The extended visibility provided by the PS FIFO improves the accuracy of

TABLE II
BASELINE BENCHMARK SUITE DESCRIPTION.

Application	Data Set	Benchmark	Effective-method	Type
bfs	`graph_19.8M.txt`	Rodinia	L1D	Mem-bound
hotspot	`18432 2 2 temp_18432_218MB.txt power_18432_184MB.txt output.out`	Rodinia	L1D	Mem-bound
srad-v1	`2000 0.5 10040 458`	Rodinia	both	Mem-bound
histo	`Img_1_109MB.bin -o ref_1.bmp -- 20 4`	Parboil	both	Mem-bound
sad	`-i reference_1_64MB.bin frame_1_64MB.bin -o out.bin`	Parboil	MSHR	Mem-bound
spmv	`Dubcova3_1_540MB.mtx.bin, vector_1_4.5MB.bin -o Dubcova3_1.mtx.out`	Parboil	MSHR	Mem-bound
lib	`Entities 100, Resolution 512,Simulation Paths 10000(Parameters in code)`	IsPass-2009	both	Mem-bound
lps	`--nx=640 --ny=512 --nz=256 --repeat=100 (Parameters in code)`	IsPass-2009	L1D	Mem-bound
2d-convolution	`NI 40960 NJ 40960 (Parameters in code)`	Polybench	L1D	Mem-bound
gembench	`inference half 5124 1500 2048 0 0`	DeepBench	L1D	Mem-bound
ray	`4096 4096`	IsPass-2009	None	Com-bound
nn	`280`	IsPass-2009	None	Com-bound
sto	`TEST_MEM_SIZE_OVERLAP 19.25MB (Parameters in code)`	IsPass-2009	None	Com-bound
convbench	`inference half 341 79 32 2 32 10 5 0 0 2 2`	DeepBench	None	Com-bound
rnnbench	`inference half 1536 1 750 gru`	DeepBench	None	Com-bound

inter-SM sharing detection, allowing the MSHR filter to more precisely identify LLC-friendly data. Consequently, higher-quality sharing decisions further increase the LLC hit rate, which in turn reduces MSHR pressure even more. This positive feedback loop enables Lazy-LLC to achieve substantial improvements in LLC hit rate and overall performance with only minimal modifications to the existing MSHR structure.

B. LLC Bandwidth Analysis

Figure 17 illustrates the improvements in bandwidth efficiency of the LLC across various implementations, normalized to the baseline. These implementations include Lazy-LLC, Lazy-LLC_MSHR, Lazy-LLC_L1D, Morpheus, AdaptCache, and an ideal case. We define bandwidth efficiency as the ratio of LLC bandwidth in Lazy-LLC relative to this baseline bandwidth, which is calculated by measuring the total data transferred through LLC partitions over a given period. This involves counting the total responses, multiplying by response size, and then dividing the transfer amount by the total simulation time.

The Lazy-LLC_MSHR and Lazy-LLC_L1D filters exhibit average improvements of 19.04% and 25.92% in memory-bound applications, leading to an overall 32.54% boost in bandwidth efficiency for Lazy-LLC. This enhancement is largely attributed to the notable reduction in LLC miss rates achieved by Lazy-LLC (see Section V-A), which reduces the frequency of accesses to slower off-chip memory, accelerating data retrieval, and allows more bandwidth for future requests. Furthermore, the improved bandwidth efficiency in Lazy-LLC results from a reduced input rate to the LLC, as non-reusable or non-shared data are filtered out. This leads to less congestion and contention within the cache, allowing for increased effective bandwidth for useful data transfers. Similar to the LLC miss rate analysis, compute-bound applications show only a minor increase in bandwidth efficiency, around 11.76%. However, Lazy-LLC outperforms both Morpheus, the current state-of-the-art model, and AdaptCache, enhancing bandwidth efficiency by approximately 21.76% and 47.36%, respectively, in memory-bound applications.

C. Interconnection Network Analysis

In Lazy-LLC, there is a strategic initiative to increase the traffic on the L1D-LLC interconnection network by caching data evicted from L1D to LLC. This tactic, combined with enhancements in LLC bandwidth on the Network on Chip (NoC), leads to a considerable increase in traffic. Figure 18 compares the traffic on the L1D-LLC interconnection network in Lazy-LLC to a baseline. Our observations indicate that Lazy-LLC increases average traffic by 12.84%. We accept this increase as a trade-off for reducing more costly off-chip traffic, as NoC traffic is more energy-efficient and bandwidth. Increasing NoC bandwidth is more economical than expanding off-chip traffic capabilities in future GPU designs. By optimizing the traffic flow between the LLC and DRAM, the GPU achieves improved memory performance and higher data transfer efficiency, which enhances computational throughput. For more details on off-chip memory accesses, please refer to Section V-D.

D. Off-chip Memory Access Analysis

The selective loading of data into the LLC ensures that the most LLC-friendly data resides in this cache in Lazy-LLC (see Section III). As a result, many of the accesses from SMs are serviced directly by the LLC, reducing the need to access off-chip memory. Furthermore, the input rate to the LLC is significantly decreased, allowing data that needs to be written back to the LLC to remain there longer. These factors together lead to a notable reduction in off-chip memory accesses. This is demonstrated in Figure 20, which shows that Lazy-LLC achieves a 15.79% reduction in total memory accesses compared to the baseline. To further analyze the impact on off-chip traffic, we separately examine write requests. As shown in Figure 19, Lazy-LLC reduces off-chip write traffic by a average of 15.53%. This reduction in writes is especially meaningful because it directly alleviates the increase in dirty evictions caused by LLC thrashing, which was highlighted in our motivation (Section II).

E. Performance Analysis

Figure 21 shows the normalized IPC of Lazy-LLC and its variants compared to Morpheus, AdaptCache, and an ideal

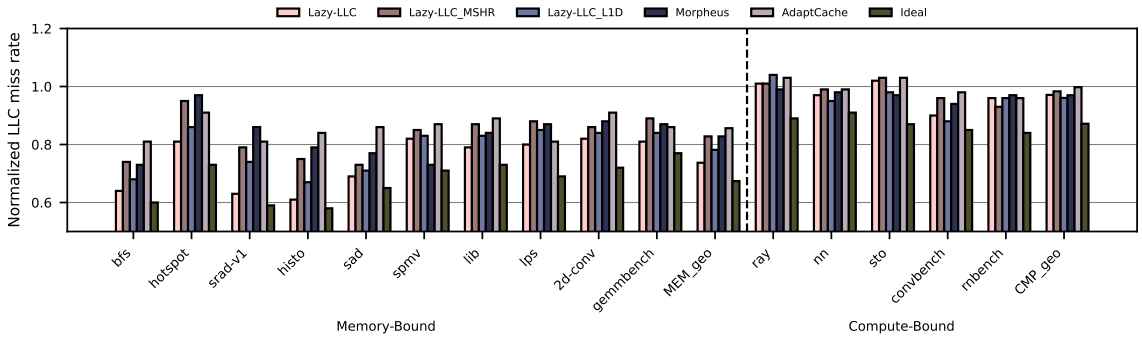


Fig. 15. Total LLC miss rate using Lazy-LLC and comparison points normalized to baseline.

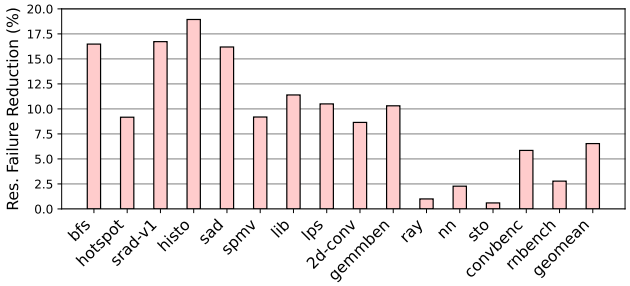


Fig. 16. Reservation failure reduction by Lazy-LLC.

bypassing scheme. In memory-bound applications, Lazy-LLC consistently delivers strong performance gains, achieving up to 48% improvement in workloads such as *histo* and *sad*, with an average speedup of 27.38% over the baseline. The combination of MSHR-based sharing detection and L1D-based reuse filtering enables Lazy-LLC to outperform both Morpheus and AdaptCache, while approaching ideal bypassing performance in several cases. In contrast, compute-bound applications exhibit only marginal gains (approximately 5%), as their working sets are small and often fit within the LLC. To avoid unnecessary overhead in such workloads, we employ a dual-threshold mechanism that dynamically activates Lazy-LLC. Following the top-down performance analysis methodology proposed by Yasin [31], we monitor the memory controller queue occupancy and enable Lazy-LLC when it exceeds 70% of the maximum queue capacity. Lazy-LLC remains active until the occupancy drops below 40%, preventing frequent on/off switching due to short-term fluctuations. This approach allows Lazy-LLC to be selectively activated for memory-bound phases while remaining disabled for compute-bound applications. Overall, Lazy-LLC outperforms Morpheus by 11.64% and AdaptCache by 14.41% in memory-bound workloads.

F. Energy Analysis

Figure 22 shows the GPU energy consumption of various implementations. These implementations include Lazy-LLC, Lazy-LLC$_{\text{MSHR}}$, Lazy-LLC$_{\text{L1D}}$, Morpheus, AdaptCache, and Ideal case. Since accessing off-chip memory is energy-intensive, minimizing these accesses can greatly reduce energy consumption. In Lazy-LLC, the LLC MSHR and L1D filter help reduce off-chip memory accesses (see Sections V-A and V-D), leading to significant energy savings. On average, Lazy-LLC_MSHR and Lazy-LLC_L1D decrease GPU energy consumption by 12.47% and 15.72%, respectively, for memory-bound GPGPU applications, resulting in a total reduction of 21.67% with Lazy-LLC. However, it's worth noting that Lazy-LLC may increase energy usage in compute-bound GPGPU applications due to lower performance in these applications (see Section V-E). Nevertheless, Lazy-LLC outperforms Morpheus and AdaptCache in terms of energy consumption, achieving reductions of 14.05% and 12.54%, respectively, in memory-bound applications.

G. Analyzing Lazy-LLC Performance Sensitivity

To evaluate the impact of Lazy-LLC's L1D associativity, LLC size, and the role of PS FIFO in performance enhancement, we analyze memory-bound applications under different GPU configurations. Figure 23 presents the normalized IPC improvement of Lazy-LLC across various setups, including *32-way* and *4-way* L1D associativity, *4*$\times$ and $\frac{1}{4}\times$ LLC size configurations, Lazy-LLC without PS FIFO, and the baseline GPU. We make four key observations. First, Lazy-LLC improves performance across all configurations, achieving an average improvement of 24%, 19%, 11%, 39%, 24%, and 27% respectively, even when L1D associativity is reduced, leading to decreased L1D filtering accuracy. Second, despite the reduction in filtering accuracy due to lower L1D associativity, Lazy-LLC remains beneficial, demonstrating its robustness. Third, while Lazy-LLC performs well with a smaller LLC ($\frac{1}{4}\times$), it continues to enhance performance even with a 4$\times$ larger LLC, highlighting its adaptability to varying cache sizes. Lastly, PS FIFO contributes a modest 3% performance boost to Lazy-LLC, but it significantly benefits MSHR-sensitive applications like SAD, improving their performance by up to 7%.

VI. Related Work

We discuss four main categories, including bypassing and throttling mechanisms, enhancing LLC capacity mechanisms, A100 persistent memory, and LLC management in CPUs.

Bypassing and Throttling Mechanisms. Thread throttling and cache bypassing are common techniques to reduce cache pressure on GPUs. Thread throttling limits the number of concurrent GPU threads [11], [32]–[38], while cache bypassing allows direct access to main memory. Most bypassing

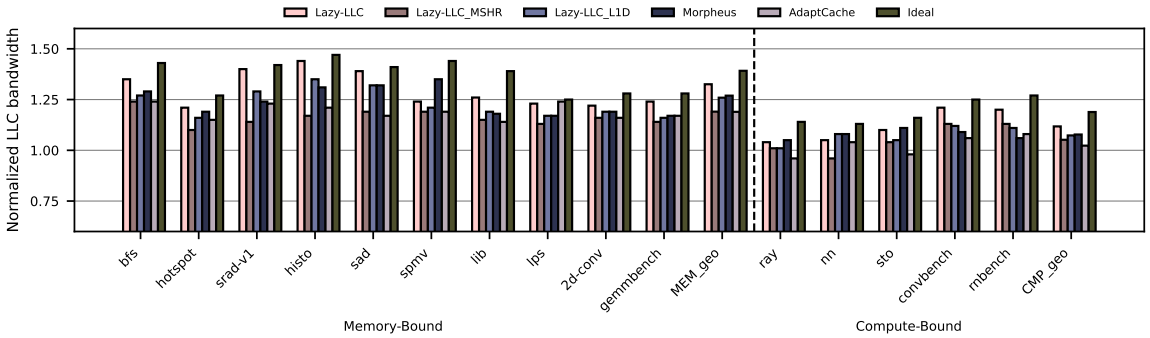


Fig. 17. Total LLC bandwidth efficiency improvement using Lazy-LLC and comparison points normalized to baseline.

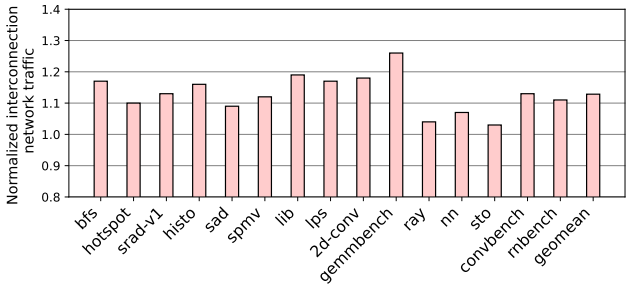


Fig. 18. Normalized nterconnection network traffic.

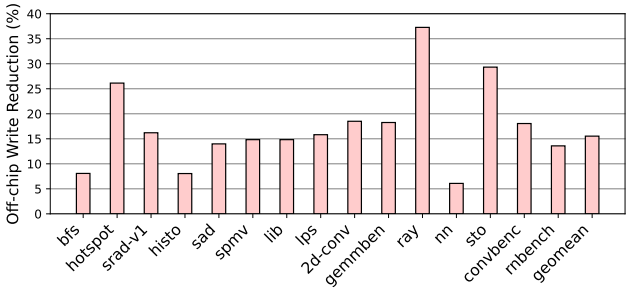


Fig. 19. Write reduction achieved by Lazy-LLC.

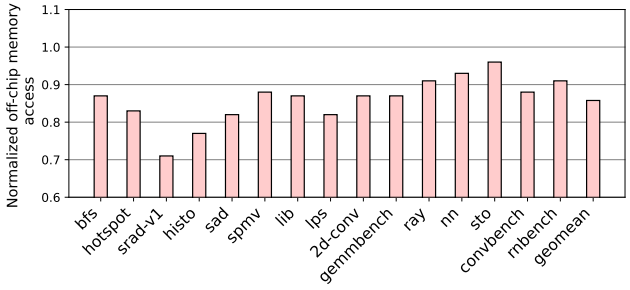


Fig. 20. Normalized Off-chip memory access.

techniques focus on L1D caches [12], [39]–[43], with limited work addressing LLCs [12]. Wang et al. [36] propose Memory Occlusion Aware Warp Scheduling (OAWS) to predict cache misses and prevent memory occlusion. They present static and dynamic OAWS versions, with the dynamic scheme balancing warp concurrency and resource utilization. Static methods offer lower accuracy due to pre-execution analysis, while dynamic approaches improve accuracy but increase hardware complexity. Li et al. [12] propose a compile-time framework that adaptively bypasses caches based on access patterns, reducing cache thrashing and improving performance for cache-sensitive applications. Although effective across GPU platforms, their work primarily targets L1D caches, leaving LLC bypassing underexplored.

Enhancing the LLC Capacity Mechanisms Methods to enhance LLC capacity include using advanced memory technologies like STT-RAM instead of SRAM or exploiting unused resources [44]–[49]. Samavatian et al. [44] propose STT-RAM for GPU LLCs, using a dual cache structure (LR and HR) with different retention times to manage frequently and infrequently written data. Darabi et al. [11] introduce Morpheus, which extends the GPU LLC by utilizing on-chip memory. However, our evaluations show that Lazy-LLC outperforms Morpheus in both performance and energy efficiency.

A100 LLC optimization NVIDIA's A100 and next-generation GPUs introduce optimizations like increased LLC capacity, LLC compression, and persistent memory to address thrashing issues, especially for modern applications such as AI that require large working datasets [8]. Persistent memory specifically targets GPU thrashing problems, while Lazy-LLC offers significant advantages over A100's LLC persistency controls through its automated, dynamic cache management. Unlike the A100, which requires manual programmer intervention to specify which data should remain in the LLC, Lazy-LLC automatically identifies LLC-friendly data using runtime information from LLC MSHRs and the L1D cache. This dynamic approach allows Lazy-LLC to adapt to changing data access patterns during execution, whereas the A100's static method relies on pre-defined persistent regions that may not efficiently handle dynamic workloads. The key difference lies in Lazy-LLC shifting the optimization burden from programmers to hardware, offering a scalable solution that mitigates LLC thrashing without adding programming complexity through explicit cache residency management API calls.

LLC management in CPUs There is extensive research on improving LLCs in CPUs [50]–[54]. While many of these approaches can be effective for thrashing in GPUs, two key differences between the architectures must be considered. First, CPU caches experience less contention due to fewer active threads, while GPUs support hundreds of threads, leading to higher cache contention. This high insertion rate in the LLC

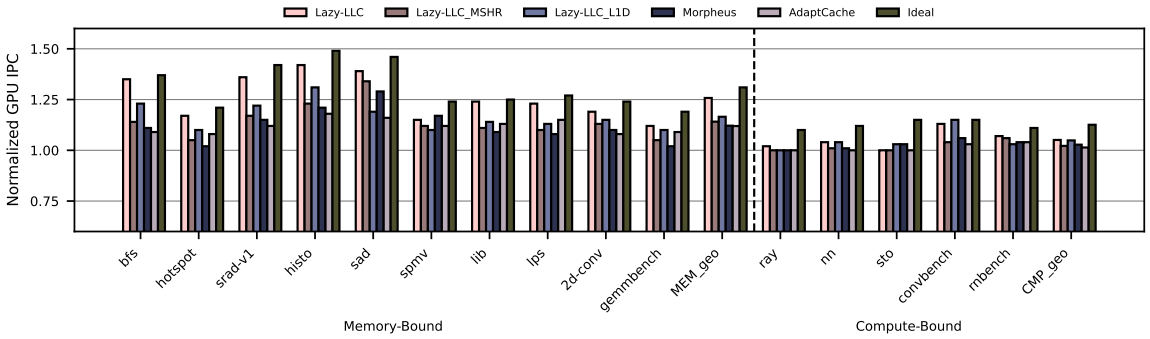


Fig. 21. Total GPU IPC using Lazy-LLC and comparison points normalized to the baseline.

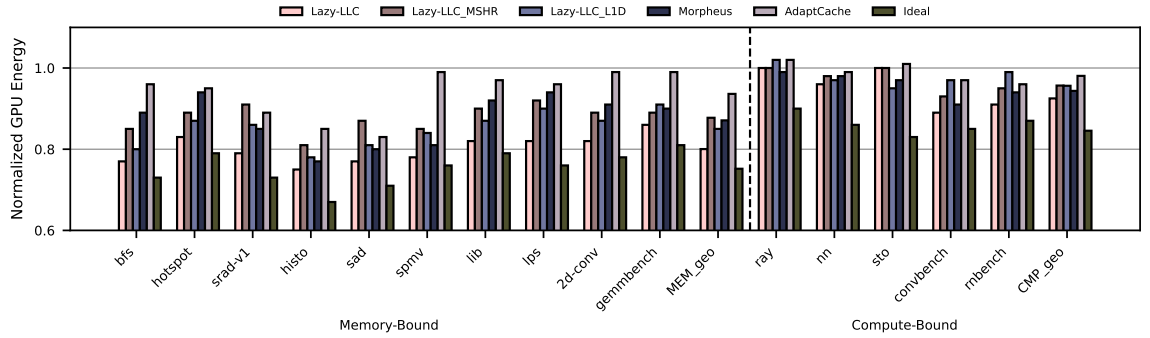


Fig. 22. Total GPU energy improvement using Lazy-LLC and comparison points normalized to the baseline.

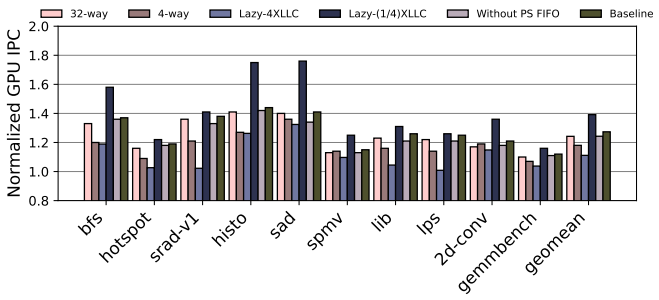


Fig. 23. Lazy-LLC Performance Sensitivity.

increases dirty data evictions, resulting in increased write traffic to off-chip memory. To address this, Lazy-LLC postpones insertion when data with locality and cache-friendly behavior is detected by the LLC-MSHR and L1D filters. Additionally, Lazy-LLC benefits from low hardware requirements by utilizing existing GPU hardware without the need for significant modifications. The third difference is that CPUs support fewer MSHR entries due to lower memory-level parallelism and bandwidth. Lazy-LLC leverages this feature by introducing an MSHR-based filter with minimal hardware changes. Furthermore, CPU-based approaches could serve as an orthogonal enhancement to Lazy-LLC, helping to improve insertion accuracy while maintaining the low hardware overhead.

VII. Conclusion

In this paper, we highlighted the significant challenge posed by LLC pressure in GPU cache management, leading to high thrashing rates and its subsequent impacts on performance. Through our analysis, we identified that existing GPU components, such as LLC MSHRs and L1D caches, can effectively pinpoint shared and reusable data. This capability allows for the selective loading of LLC-friendly data, directly reducing LLC pressure and thereby improving GPU performance without substantial overhead. Our proposed Lazy-LLC technique leverages these components to distinguish between shared, reusable, and non-reusable data, ensuring only LLC-friendly data is cached in the LLC. This approach not only mitigates LLC pressure but also enhances overall system performance. The experimental results demonstrate that Lazy-LLC significantly reduces LLC miss rates by 27.03%, increases effective bandwidth by 32.54%, and achieves a 27.38% improvement in overall performance and a 21.67% enhancement in energy efficiency compared to existing methods, with minimal overhead.

References

[1] H. Kim, R. Vuduc, S. Baghsorkhi, J. Choi, and W.-m. Hwu, "Performance analysis and tuning for general purpose graphics processing units (gpgpu)," *Synthesis Lectures on Computer Architecture*, vol. 7, no. 2, pp. 1–96, 2012.

[2] T. M. Aamodt, W. W. L. Fung, and T. G. Rogers, "General-purpose graphics processor architectures," *Synthesis Lectures on Computer Architecture*, vol. 13, no. 2, pp. 1–140, 2018.

[3] Z. Zhang, M. Naderan-Tahan, M. Jahre, and L. Eeckhout, "Sac: Sharing-aware caching in multi-chip gpus," 2023.

[4] X. Zhao, A. Adileh, Z. Yu, Z. Wang, A. Jaleel, and L. Eeckhout, "Adaptive memory-side last-level gpu caching," in *Proceedings of the 46th international symposium on computer architecture*, 2019, pp. 411–423.

[5] M. Ma and Z. Liu, "Llc access behavior prediction method for multi-core processors," 2022.

[6] Z. Wu, W. Chen, B. Li, Z. Liu, and S. Long, "Deadblock aware adaptive eviction policy for shared last-level cache," 2022.

[7] NVIDIA, "Nvidia volta architecture," NVIDIA Corporation, Tech. Rep., 2017, accessed: 2024-11-11. [Online]. Available: https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

[8] NVIDIA Corporation, "Nvidia ampere architecture whitepaper," 2020, available at https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf.

[9] T. Sorensen, S. Pai, and A. F. Donaldson, "One size doesn't fit all: Quantifying performance portability of graph applications on gpus," 2019.

[10] X. Zhao, M. Jahre, and L. Eeckhout, "Selective replication in memory-side gpu caches," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 967–980.

[11] S. Darabi, M. Sadrosadati, N. Akbarzadeh, J. Lindegger, M. Hosseini, J. Park, J. Gómez-Luna, O. Mutlu, and H. Sarbazi-Azad, "Morpheus: Extending the last level cache capacity in gpu systems using idle gpu core resources," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 228–244.

[12] A. Li, G.-J. van den Braak, A. Kumar, and H. Corporaal, "Adaptive and transparent cache bypassing for gpus," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015, pp. 1–12.

[13] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 473–486.

[14] Y. Cai and S. See, *GPU computing and applications*. Springer, 2015.

[15] A. Choudhury, B. Mondal, K. Paul, and B. K. Sikdar, "Llc block reuse predictor design using deep learning to mitigate soft error in multicore," pp. 690–695, 2024.

[16] A. Sharma, S. Kondguli, and M. C. Huang, "Irrelevant data traffic in modern low power gpu architectures," pp. 1–7, 2022.

[17] N. Corporation. (2019) Geforce rtx 2060 super user guide. [Online]. Available: https://www.nvidia.com/content/geforce-gtx/GEFORCE_RTX_2060_SUPER_User_Guide.pdf

[18] Accel-Sim Project, "Gpgpu-sim tested configuration files," https://github.com/accel-sim/gpgpu-sim_distribution/tree/dev/configs/tested-cfgs, 2026, accessed: 2026-06-16.

[19] Y. Gu and L. Chen, "Dynamically linked mshrs for adaptive miss handling in gpus," in *Proceedings of the ACM International Conference on Supercomputing*, 2019, pp. 510–521.

[20] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi, "CACTI 6.0: A tool to model large caches," *HP laboratories*, 2009.

[21] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, "Accelwattch: A power modeling framework for modern gpus," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 738–753.

[22] J. Karimov, T. Rabl, and V. Markl, "Polybench: The first benchmark for polystores," in *Performance Evaluation and Benchmarking for the Era of Artificial Intelligence: 10th TPC Technology Conference, TPCTC 2018, Rio de Janeiro, Brazil, August 27–31, 2018, Revised Selected Papers 10*. Springer, 2019, pp. 24–41.

[23] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *2009 IEEE international symposium on workload characterization (IISWC)*. Ieee, 2009, pp. 44–54.

[24] A. Bakhoda, G. L. Yuan, W. W. Fung, H. Wong, and T. M. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," in *2009 IEEE international symposium on performance analysis of systems and software*. IEEE, 2009, pp. 163–174.

[25] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W.-m. W. Hwu, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," *Center for Reliable and High-Performance Computing*, vol. 127, p. 27, 2012.

[26] Q. Zhang, L. Zha, J. Lin, D. Tu, M. Li, F. Liang, R. Wu, and X. Lu, "A survey on deep learning benchmarks: Do we still need new ones?" in *Benchmarking, Measuring, and Optimizing: First BenchCouncil International Symposium, Bench 2018, Seattle, WA, USA, December 10-13, 2018, Revised Selected Papers 1*. Springer, 2019, pp. 36–49.

[27] S. Narang, "Deepbench," https://svail.github.io/DeepBench/, 2016, baidu Research, Silicon Valley AI Lab (SVAIL).

[28] B. Hu and C. J. Rossbach, "Altis: Modernizing gpgpu benchmarks," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020, pp. 1–11.

[29] M. Naderan-Tahan and L. Eeckhout, "Cactus: Top-down gpu-compute benchmarking using real-life applications," in *IEEE International Symposium on Workload Characterization (IISWC)*, 2021, pp. 176–188.

[30] L. Zhang, M. Wahib, P. Cheng, J. Meng, X. T. Wang, T. Endo, and S. Matsuoka, "Perks: a locality-optimized execution model for iterative memory-bound gpu applications," 2022.

[31] A. Yasin, "A Top-Down Method for Performance Analysis and Counters Architecture," in *ISPASS*, 2014.

[32] T. G. Rogers, M. O'Connor, and T. M. Aamodt, "Cache-conscious wavefront scheduling," in *2012 45th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2012, pp. 72–83.

[33] ——, "Divergence-Aware Warp Scheduling," in *MICRO*, 2013.

[34] H. Wang, F. Luo, M. Ibrahim, O. Kayiran, and A. Jog, "Efficient and fair multi-programming in gpus via effective bandwidth management," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 247–258.

[35] O. Kayıran, A. Jog, M. T. Kandemir, and C. R. Das, "Neither more nor less: Optimizing thread-level parallelism for gpgpus," in *Proceedings of the 22nd international conference on Parallel architectures and compilation techniques*. IEEE, 2013, pp. 157–166.

[36] B. Wang, Y. Zhu, and W. Yu, "Oaws: memory occlusion aware warp scheduling," in *2016 International Conference on Parallel Architecture and Compilation Techniques (PACT)*. IEEE, 2016, pp. 45–55.

[37] D. Li, M. Rhu, D. R. Johnson, M. O'Connor, M. Erez, D. Burger, D. S. Fussell, and S. W. Redder, "Priority-based cache allocation in throughput processors," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2015, pp. 89–100.

[38] M. Mao, J. Hu, Y. Chen, and H. Li, "Vws: A versatile warp scheduler for exploring diverse cache localities of gpgpu applications," in *2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*. IEEE, 2015, pp. 1–6.

[39] X. Chen, L.-W. Chang, C. I. Rodrigues, J. Lv, Z. Wang, and W.-M. Hwu, "Adaptive cache management for energy-efficient gpu computing," in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2014, pp. 343–355.

[40] W. Jia, K. A. Shaw, and M. Martonosi, "Characterizing and improving the use of demand-fetched caches in gpus," in *Proceedings of the 26th ACM international conference on Supercomputing*, 2012, pp. 15–24.

[41] X. Xie, Y. Liang, Y. Wang, G. Sun, and T. Wang, "Coordinated static and dynamic cache bypassing for gpus," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2015, pp. 76–88.

[42] C. Li, S. L. Song, H. Dai, A. Sidelnik, S. K. S. Hari, and H. Zhou, "Locality-driven dynamic gpu cache bypassing," in *Proceedings of the 29th ACM on International Conference on Supercomputing*, 2015, pp. 67–77.

[43] W. Jia, K. A. Shaw, and M. Martonosi, "Mrpb: Memory request prioritization for massively parallel processors," in *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 272–283.

[44] M. H. Samavatian, H. Abbasitabar, M. Arjomand, and H. Sarbazi-Azad, "An efficient stt-ram last level cache architecture for gpus," in *Proceedings of the 51st Annual Design Automation Conference*, 2014, pp. 1–6.

[45] M. Gebhart, S. W. Keckler, B. Khailany, R. Krashinsky, and W. J. Dally, "Unifying primary cache, scratch, and register file memories in a throughput processor," in *2012 45th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2012, pp. 96–106.

[46] Y. Oh, G. Koo, M. Annavaram, and W. W. Ro, "Linebacker: Preserving victim cache lines in idle register files of gpus," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 183–196.

[47] N. Jing, J. Wang, F. Fan, W. Yu, L. Jiang, C. Li, and X. Liang, "Cache-emulated register file: An integrated on-chip memory architecture for high performance gpgpus," in *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–12.

[48] R. Komuravelli, M. D. Sinclair, J. Alsop, M. Huzaifa, M. Kotsifakou, P. Srivastava, S. V. Adve, and V. S. Adve, "Stash: Have your scratchpad and cache it too," *ACM SIGARCH Computer Architecture News*, vol. 43, no. 3S, pp. 707–719, 2015.

[49] B. Li, J. Wei, J. Sun, M. Annavaram, and N. S. Kim, "An efficient gpu cache architecture for applications with irregular memory access patterns," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 16, no. 3, pp. 1–24, 2019.

[50] J. Gaur, M. Chaudhuri, and S. Subramoney, "Bypass and insertion algorithms for exclusive last-level caches," in *Proceedings of the 38th annual international symposium on Computer architecture*, 2011, pp. 81–92.

[51] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Adaptive insertion policies for high performance caching," *ACM SIGARCH Computer Architecture News*, vol. 35, no. 2, pp. 381–391, 2007.

[52] C.-J. Wu, A. Jaleel, M. Martonosi, S. C. Steely Jr, and J. Emer, "Pacman: prefetch-aware cache management for high performance caching," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, 2011, pp. 442–453.

[53] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely Jr, and J. Emer, "Ship: Signature-based hit predictor for high performance caching," in *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture*, 2011, pp. 430–441.

[54] D. A. Jiménez, "Insertion and promotion for tree-based pseudolru last-level caches," in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, 2013, pp. 284–296.